

# Глубокий разбор: Градиентный бустинг над решающими деревьями (GBDT)

Твой персональный гид по ML

4 мая 2026 г.

## Введение

Градиентный бустинг над решающими деревьями (GBDT) — это своего рода «тяжелая артиллерия» в классическом машинном обучении. Давай разберем этот алгоритм глубже, заглянув под капот математики и практических нюансов.

## Содержание

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Философское отличие: Бустинг vs Бэггинг</b>                     | <b>2</b> |
| <b>2</b> | <b>Математика: Градиентный спуск в функциональном пространстве</b> | <b>2</b> |
| <b>3</b> | <b>Секрет успеха: Почему деревья неглубокие?</b>                   | <b>3</b> |
| <b>4</b> | <b>Борьба с «жадностью»: Регуляризация</b>                         | <b>3</b> |
| <b>5</b> | <b>Тройка лидеров: XGBoost, LightGBM, CatBoost</b>                 | <b>3</b> |
| 5.1      | CatBoost (от Яндекса) . . . . .                                    | 3        |
| 5.2      | LightGBM (от Microsoft) . . . . .                                  | 3        |
| 5.3      | XGBoost . . . . .                                                  | 4        |
| <b>6</b> | <b>Интерпретация: Feature Importance</b>                           | <b>4</b> |

# 1 Философское отличие: Бустинг vs Бэггинг

Чтобы понять бустинг, нужно вспомнить случайный лес (бэггинг).

- **Бэггинг** (Random Forest) строит много глубоких, умных деревьев **независимо** и усредняет их. Это борется с **разбросом** (variance).
- **Бустинг** строит цепочку из очень простых («глупых») деревьев **последовательно**. Каждое следующее дерево исправляет ошибки всех предыдущих. Это борется со **смещением** (bias).

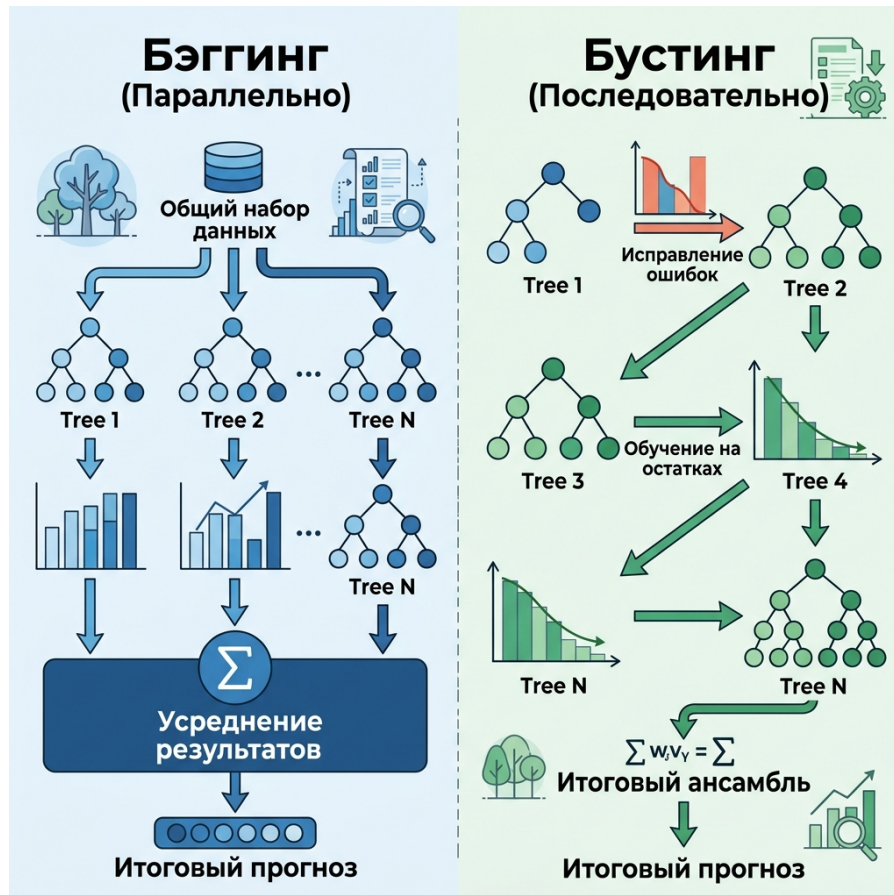


Рис. 1: Концептуальное различие между Бустингом и Бэггингом.

## 2 Математика: Градиентный спуск в функциональном пространстве

В градиентном бустинге мы меняем **саму функцию**  $a(x)$ .

Представь, что текущая композиция  $a_{k-1}$  выдает предсказания. Мы хотим добавить к ней дерево  $b_k$ , чтобы общая ошибка упала.

1. **Градиент:**  $g_i = \frac{\partial \mathcal{L}(y_i, z)}{\partial z}$ . Это число говорит, куда нужно «подвинуть» предсказание.
2. **Антиградиент:** Нам нужно идти в противоположную сторону ( $-g_i$ ).
3. **Обучение на остатки:** Новое дерево  $b_k$  обучается предсказывать значения  $-g_i$ .

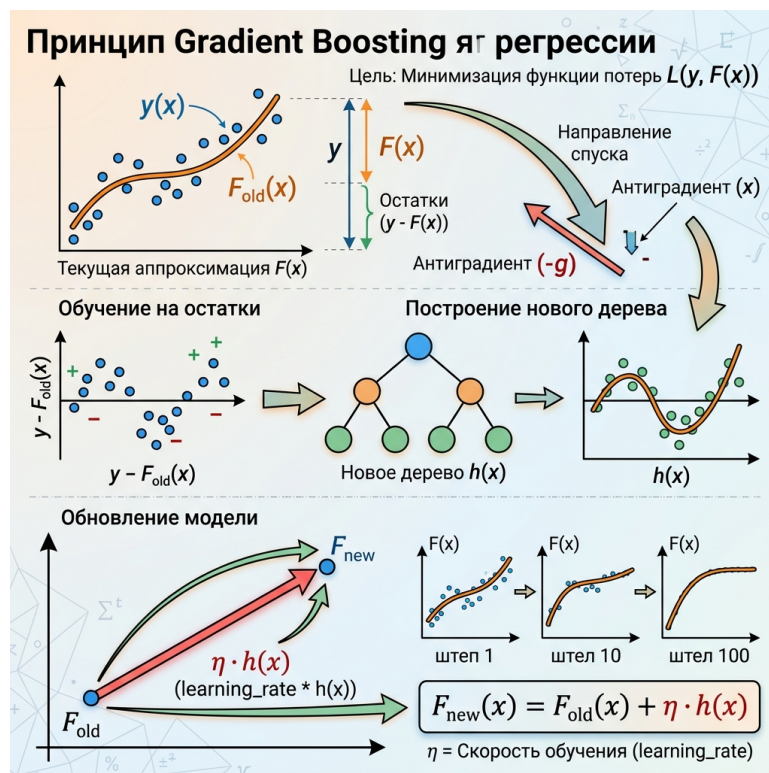


Рис. 2: Процесс обучения нового дерева на антиградиент функции потерь.

### 3 Секрет успеха: Почему деревья неглубокие?

В GBDT базовые деревья обычно имеют глубину всего **3–6 уровней**. Смысл бустинга в том, чтобы каждое дерево вносило **крошечный, но точный вклад**. Тысяча маленьких уточнений работают лучше, чем пять больших и грубых.

### 4 Борьба с «жадностью»: Регуляризация

- **Learning Rate (Shrinkage)**: Мы берем лишь малую часть предсказаний нового дерева (например, 0.1). Это делает алгоритм устойчивым к шуму.
- **Стохастичность**: Использование случайных подмножеств данных для построения каждого дерева (Stochastic Gradient Boosting).

### 5 Тройка лидеров: XGBoost, LightGBM, CatBoost

#### 5.1 CatBoost (от Яндекса)

- **Симметричные деревья**: Все узлы на одном уровне используют один и тот же признак.
- **Категориальные признаки**: Продвинутое кодирование "из коробки".

#### 5.2 LightGBM (от Microsoft)

- **Leaf-wise рост**: Выбирает самый «перспективный» лист и разбивает его.
- **Гистограммный метод**: Сжатие признаков в «корзины» для ускорения.

# Сравнение структур деревьев



Рис. 3: Различия в структуре деревьев: CatBoost (симметричные) vs LightGBM (полистовые).

## 5.3 XGBoost

- **Вторая производная:** Использует разложение Тейлора до второго порядка (Гесс-сиан) для точности.

## 6 Интерпретация: Feature Importance

1. **Weight (Frequency):** Частота использования признака.
2. **Gain:** Суммарное падение ошибки от признака.
3. **Permutation Importance:** Падение качества при перемешивании значений.

Главное при работе с бустингом — следить за графиком лосса на валидации. Как только ошибка на валидации перестала падать, а на трейне продолжает — немедленно останавливайся (это называется *Early Stopping*).

**Antigravity:** Для большинства табличных задач CatBoost — это выбор по умолчанию благодаря отличной работе с категориями и регуляризации.